Tri-Cam: Practical Eye Gaze Tracking via Camera Network

Sikai Yang
syang126@ucmerced.edu
University of California, Merced
Merced, CA, USA

arXiv:2409.19554v1 [cs.CV] 29 Sep 2024

ABSTRACT

As human eyes serve as conduits of rich information, unveiling emotions, intentions, and even aspects of an individual's health and overall well-being, gaze tracking also enables various human-computer interaction applications, as well as insights in psychological and medical research. However, existing gaze tracking solutions fall short at handling free user movement, and also require laborious user effort in system calibration. We introduce Tri-Cam, a practical deep learning-based gaze tracking system using three affordable RGB webcams. It features a split network structure for efficient training, as well as designated network designs to handle the separated gaze tracking tasks. Tri-Cam is also equipped with an implicit calibration module, which makes use of mouse click opportunities to reduce calibration overhead on the user's end. We evaluate Tri-Cam against Tobii, the state-of-the-art commercial eye tracker, achieving comparable accuracy, while supporting a wider free movement area. In conclusion, Tri-Cam provides a user-friendly, affordable, and robust gaze tracking solution that could practically enable various applications.

1 INTRODUCTION

Gaze tracking refers to estimating where the user is looking. Aside from existing human-computer interaction methods, such as gesture recognition [1–4] and haptic interaction [5], gaze tracking brought new possibility of using human eyes as a new way of interaction [6–8]. As eyes convey a wealth of information about emotions, intentions, and health, eye gaze tracking enables various applications in smart interaction [9–16], psychology [17–19], and medicine [12, 20–22].

In this paper, we target the monitor-based gaze tracking, namely to infer where the user is looking on the screen. It is a kind of remote gaze tracking, where the cameras are stationary, while the user is moving. It has tremendous application market as people use computers and look at screens everyday.

Free movement and depth perception: Remote gaze tracking poses a great challenge to the interaction systems, due to free movement, namely, the relative location between the user's eyes and the cameras changes. It necessitates depth perception capability for the gaze tracking systems to support practical and user-friendly gaze tracking applications. Single camera [23, 24] cannot effectively handle depth, consequently demanding minimal user movement. Although facial features may support depth perception [25], in gaze tracking applications, users may frequently turn their head to look in various directions, making the solution less reliable. Several works propose to use depth camera to handle free movement [26, 27]. The Tobii eye trackers [28] use infrared sensors to capture unique reflection from human eyes, and can be robust to distance. However, these hardwares are relatively expensive. A practical solution is to deploy multiple inexpensive non-depth cameras [29–32]. It leverages geometric relationship between the cameras to handle depth information, similar to the reason that human has two eyes. However, too many cameras maybe also pose affordability issue [33].

In this work, we develop a practical gaze tracking system, Tri-Cam. It utilizes three non-depth RGB webcams (each costing $10) to handle free movement. Compared to double-camera systems [29, 32], triple-camera enables an exclusive intra-validation mechanism via auxiliary multitasking. It further exploits geometrical relationships between cameras to enhance gaze tracking accuracy.

Split gaze tracking neural network: The key to handling free movement lies in understanding the geometry between cameras, eyes, and the screen. We analyze the gaze tracking task and divide it into camera-eye geometry and eye-screen geometry. Accordingly, we design a split neural network structure that handles each geometry separately. This division allows each part of the network to focus on its designated task, resulting in a lightweight network.

Camera-eye geometry and intra-validation: The camera-eye geometry refers to the vector from cameras to eyes. This information can be represented by the coordinate where the eye is detected within the camera view. Multiple camera-eye geometries form triangles that enable depth perception and consequently handle free movement. To further exploit the camera-eye geometry, we develop an intra-validation design via auxiliary multi-tasking. [34, 35] It masks one camera, and uses the other two cameras to infer the camera-eye geometry of the masked camera, resulting in extra supervision for the network training, and enhanced gaze tracking performance.

Eye-screen geometry and weighted fusion: The eye-screen geometry refers to the gaze ray from the eyes to the gaze point on the screen. This information is hidden within the eye images. We rely on convolutional neural networks to extract this information. However, eye images may have quality fluctuations due to reflection, occlusion or eye blinking. To make the

system robust to fluctuations, we design a discriminator-based weighted fusion mechanism that prioritizes higher-quality camera channels among the three cameras and two eyes, helping the system focus on relevant information.

Low-effort re-calibration via implicit data collection: Re-calibration can be a heavy burden, demanding users to look at visual targets to collect aligned gaze data for training. Several works discusses the feasibility of utilizing opportunities in passive human attention [36–38], e.g., video saliency. Sugano *et al.* [39] proposed to use mouse clicks as aligned opportunities, but assumed that the user's gaze is directed at the mouse cursor, without discussing the potential error within the opportunities. Huang *et al.* [40] conducted insightful observation on the alignment error during mouse click, taking in consideration of the size of interaction target and temporal perspective. However, they missed valuable context information that can be extracted from daily computer usage, e.g., application context and the spatial distribution of click error. To that end, we develop an implicit calibration module that utilizes aligned opportunities during mouse clicks to collect gaze data without explicit user participation. By studying daily computer usage patterns, we establish three criteria: application context, press-release duration, and spatial distribution. These criteria help filter and refine the implicitly collected data, thereby reducing the need for explicit data collection that requires active user attention. We also demonstrate the feasibility of combining implicit and explicit calibration, providing users with an option to balance between accuracy and effort.

We collected 84,000 gaze data samples from 21 participants, allowing them to move freely in front of the monitor to simulate real-life computer usage scenarios. Extensive experiments are conducted to evaluate the gaze tracking performance of Tri-Cam, primarily comparing it to the state-of-the-art commercial eye tracker, Tobii Pro Spark [28, 41]. The experiment results indicate that at a distance of 50cm from the monitor, Tri-Cam achieves an average gaze inference error of 2.06cm, which is near the 1.95cm error of Tobii eye tracker, while Tri-Cam supports a wider range of free-movement.

In summary, we make the following contributions:

- We analyze and partition the gaze tracking task into camera-eye geometry and eye-screen geometry. Accordingly, we design a light weight split neural network structure to handle the separated tasks.
- For camera-eye geometry, we propose a novel intra-validation mechanism to exploit geometric relationships between cameras and enhance gaze tracking accuracy.
- For eye-screen geometry, we devise a weighted-fusion strategy to improve robustness to image quality fluctuations.
- We develop an implicit calibration module that leverages mouse click opportunities, reducing the data collection overhead for the user. Based on real-life computer usage observation, we establish three criteria to filter and refine the implicitly collected data samples.
- We collect free-movement gaze data from 21 users and conduct extensive experiments to evaluate Tri-Cam.

2 SCENARIO SETUP

We first introduce the application scenario of remote gaze tracking, including several important technical configurations.

As an overview, in the context of remote gaze tracking, a user sits in front of a monitor and looks at the screen. Meanwhile, multiple cameras on the monitor capture the user's eye image, based on which, the gaze tracking system estimates the user's gaze point on the screen.

2.1 Cameras

The cameras are inexpensive ($10 each) webcams without depth or infrared capabilities, with a resolution of 1920×1080. We positioned three webcams on the top of the monitor, ensuring that their relative position and orientation relative to the screen remain fixed. This camera-on-monitor setup is less likely to change compared to cameras placed on a desktop, which are more prone to accidental touch or movement.

For each monitor and its associated cameras, we train a dedicated model. The implicit re-calibration module would help reduce the overhead of training new models for different monitor-camera setups. If the monitor is moved while the cameras remain fixed, the existing model does not require retraining. However, if the cameras on the monitor are significantly re-positioned, the new setup should be treated as distinct, necessitating model re-training.

2.2 Gaze Point

The gaze point indicates where the user is looking, referring to the specific pixel on the screen of 1920×1080 resolution. Ground truth for the gaze point can be obtained by displaying a visual target on the screen and instructing the user to focus on it. The coordinate of the center pixel of the target serves as the ground truth for the gaze point.

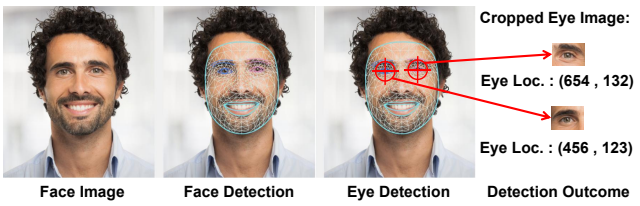

Figure 1: Eye detection via face detection

2.3 Eye Detection

The system initially performs eye detection to determine if the user is looking at the screen and then locate the eyes within

the camera's field of view. Several eye detection algorithms exist, including OpenCV Haar cascades and Dlib eye detector. In our system, we utilize the MediaPipe eye detection algorithm [42]. Figure 1 (the image is sourced from the MediaPipe website) illustrates the workflow of the eye detection module. MediaPipe first detects the face and then identifies the eyes, providing the locations of the detected eyes within the camera's view. With this information, we can extract the eye images from the face image, resulting in four outputs: two cropped eye images and their corresponding coordinates within the camera's view.

2.4 Practicality Standards

We aim to develop a practical gaze tracking system that can be widely deployed and used with ease. Therefore, we identify three important standards for evaluating the practicality of such a system: affordability, robustness, and user-friendliness.

2.4.1 Affordable Hardware. First and foremost, a practical system should be affordable. Commercial eye trackers typically range from $200 to $4000, making them accessible primarily to customers with professional interests, such as gamers and streamers. For instance, the Tobii Pro Spark, which we used as a comparison in this work, costs $3300. The gaming version, Tobii Tracker 5, is priced at $250 but is limited to gaming assistance functions only. Both devices feature a central infrared camera and two infrared emitters on the sides. Depth cameras, on the other hand, are priced around $100, making them prohibitively expensive for many users. Practical choices are often limited to non-depth RGB cameras, which are available for approximately $10 to $20.

2.4.2 Robustness to Free Movement. User movement will affect the spatial relationship between the eyes and the cameras, meaning that the eyes will appear at any position in the cameras' view. This variability can greatly impact the performance of eye gaze tracking. In this paper, to ensure practicality and user experience, we allow users to move freely. This means that users can assume any comfortable body posture, and their eyes can be oriented in any direction relative to the cameras. They are only asked to maintain a healthy distance of 45 to 60 cm away from the screen.

Depth perception poses a significant challenge for gaze tracking systems with free movement. Only with depth perception can the system project the gaze ray onto the screen to infer the location of gaze point. Since depth cameras are not considered due to affordability reasons, we utilize multiple non-depth cameras to form a camera network and exploit their geometric relationships to perceive depth information.

2.4.3 User-Friendliness. Finally, a practical system should also minimize users' effort in preparation or redeployment. Gaze tracking systems typically require calibration before they are ready to use. Specifically, for learning-based gaze tracking, calibration is accomplished by training with the new user's data. To speed up training and reduce data collection overhead, the neural network size needs to be reduced to minimize the parameters to be trained. To achieve this, we design a split neural network structure (Section 3.1) that allows each part of the network to focus on its designated task with a limited network size. Additionally, we propose an implicit data collection module for Tri-Cam (Section 3.4) to further reduce data collection overhead and achieve low-effort redeployment for new users or new camera-monitor setups.

3 DESIGN

We now introduce the system design of Tri-Cam. We utilize three non-depth cameras to perceive depth information. To jointly consider all cameras, in addition to the visual information hidden within human eye images, we rely on convolutional neural networks (CNN) to process eye images, and a multi-layer perceptron (MLP) to fuse all information.

We first analyze and divide the gaze tracking mechanism into two-fold: camera-eye geometry and eye-screen geometry. Accordingly, we design a split neural network structure to handle the two geometries. Then, to make full use of the multi-camera network, we design an intra-validation mechanism, which aims to make the model better capture the camera-eye geometry. For eye-screen geometry, we perform weighted fusion for the eye images, using a joint discriminator that judges the importance of each captured eye image. Lastly, to reduce re-calibration (re-training) overhead, we design a low-effort implicit data collection module that makes use of cursor click opportunities.

3.1 Gaze Tracking Geometry

In the context of gaze tracking, the user looks at the screen while the cameras focus on the user's eyes. We divide the gaze tracking mechanism into two steps: camera-to-eye and eye-to-screen.

3.1.1 Camera-Eye Geometry. First, the cameras capture the user's image. Then, via eye detection algorithm, the system locates user's eyes in each camera's view. The eye center's pixel within the camera's view indicates the direction of the eye relative to the camera. With multiple cameras and directions, and via triangulating, the system is capable of anticipating the position of the eye, or in another word, acquiring depth information.

3.1.2 Eye-Screen Geometry. As the eye position is acquired from the camera-eye geometry, based on that, the eye-screen geometry then infers the direction that the eye is looking in. This information is embedded within the eye image, and can be learned by convolutional neural networks.

With this direction, and from the anticipated eye position, we can project the eye sight on to the screen to infer where the the user is looking at.

3.1.3 Split Neural Network Framework. Based on the understanding of camera-eye geometry and eye-screen geometry, we design the gaze tracking neural network in a split form to separately handle the two geometries. The network structure is shown in Figure 2. For simplicity, Figure 2 only shows the processing of the right eye. The left eye will apply the same network structure, and share the final decision multi-layer perceptron to jointly output one gaze point.

In the upper part, we input the pixel-coordinates of the eyes within the camera's field of view, which are acquired via the eye detection algorithm. Specifically, if the eye is not detected in the camera's view at a certain moment, the eye location will be overwritten as -1, and the cropped image will be an all-black image when the eye is not detected.

In the lower part, we only feed it with the cropped eye image, as the eye sight information is entirely concentrated within this area. The background image contains little useful information for the task of gaze tracking and should thus be discarded. Motivated by a head-mounted eye gaze tracking work, Invisible-Eye [30], which uses low resolution but multiple images to achieve lightweight gaze direction, we also resize the resolution to 40×20, which is still clear enough to distinguish the eye movement. We use the six 40×20-resolution eye images (2 eyes × 3 cameras = 6 images) to infer the eye-screen geometry, which is relatively lightweight, and thus guarantees fast calibration and re-training.

Via splitting the architecture into two parts, each part focus on its designated task. However, since the lower part handles eye-screen geometry and processes images, which is large in dimension, we use a fully connected layer (FC) to further process its output and reduce its dimension to match the lower dimension of camera-eye geometry. Finally, in the final decision multi-layer perceptron, both pieces of information are jointly considered to calculate the gaze point.

3.2 Intra-Validation via Auxiliary Multitasking

To better master the camera-eye geometry, we design an intra-validation mechanism that leverages existing view angle information more effectively. This mechanism improves system accuracy through auxiliary multi-tasking.

3.2.1 Design Motivation. Assuming the gaze tracking system has been well calibrated and trained, based on the camera-eye geometries of two cameras, the system should be able to know the camera-eye geometry of the third camera.

Specifically, based on the coordinate of the eye in the view of two cameras, the system knows the directions from both cameras to the eye. Based on these two directions, the system can deduce the position of the eye, and consequently determine the direction in which the third camera will observe the eye, i.e., the coordinate of the eye in the third camera.

A well trained gaze tracking system should be able to handle the relationship between the camera-eye geometries from three cameras. Conversely, we expect to improve the system via guiding it to learn the relationship in the camera-eye geometries, via an auxiliary multitasking design [34].

3.2.2 Intra-Validation Network Design. Based on the above speculations, we fuse the camera-eye geometry information of camera 1 and 2, i.e., coordinate of eye in the camera's view, into fully connected layers and let it calculate the camera-eye geometry information of camera 3 as an auxiliary output. We use the actual eye coordinate in camera 3's view as ground truth to calculate the auxiliary loss. We also use camera 2 and 3 to predict the camera-eye geometry of camera 1, and use camera 1 and 3 to predict the camera-eye geometry of camera 2. The auxiliary losses of the predicted camera-eye geometries will be used to perform back propagation to train the fully connected layers at the middle in conjunction with the back propagation of the main gaze point loss. The auxiliary losses and the main loss will be combined into a joint loss. We set a ratio of 0.1 for all three auxiliary losses as they are incorporated into the joint loss. If the eye is not detected, the neural network pipeline will be multiplied by 0 before the last fully connected layer, so that the back propagation will not update the networks before it.

This architecture does not require extra information or measurement, but instead, masks and infers existing information. The mechanism behind these auxiliary inferences is pertinent to the main task of the network, and thus could facilitate the training of related parts of the network.

3.2.3 Why Three Cameras. The intra-validation design aims to harness the potential of the camera network, which is exclusive to systems with three or more cameras. With one camera's information masked, two cameras remain, allowing them to form a triangle with their geometric information.

However, if there are only two cameras, it is not practical to infer the direction in which one camera will see the eye using the other camera. Webcams can only provide directional information. As a result, the possible location of the eye can vary along the direction, leading to uncertainty in the direction of the detected eye for the other camera, as depicted in Figure 3. Therefore it would not be sensible to conduct intra-validation between the two cameras, let alone expect an improvement in system performance. While it may be feasible to measure the depth of an object with a non-depth camera using the size of the object in the image, in the task of gaze tracking, users' eye image size may often vary due to

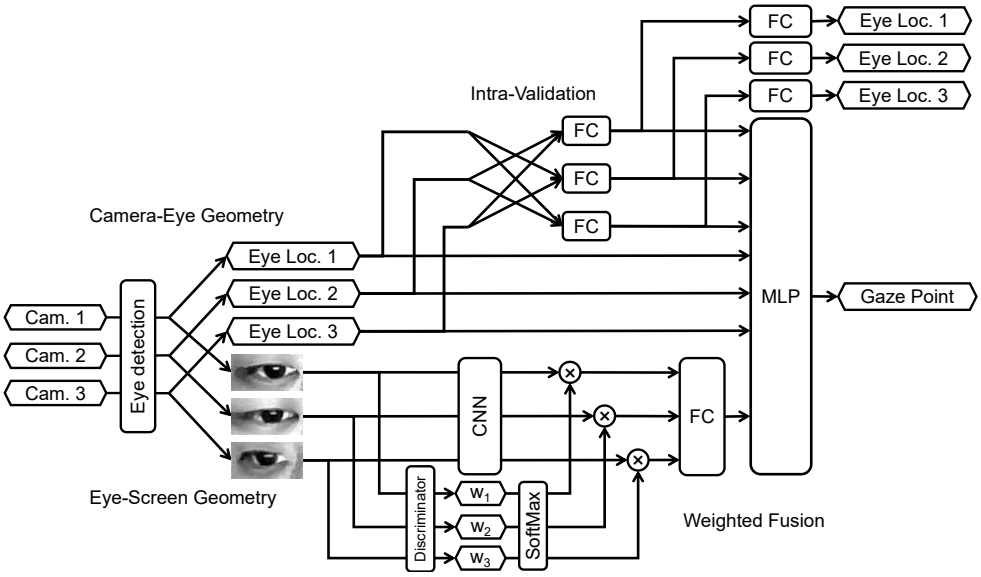

Figure 2: Tri-Cam Neural Network Architecture

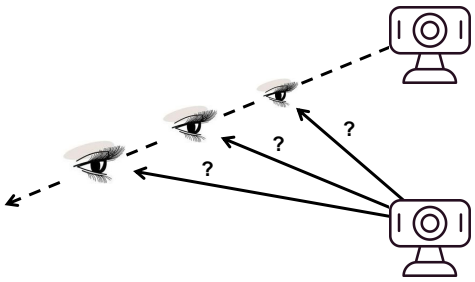

Figure 3: Two cameras are not enough to support the intra-validation mechanism

different eye openness and view angle, making it impractical to use the size of eye images to infer depth.

3.3 Weighted Camera Fusion via Joint Discriminator

The eye-screen geometry refers to the gaze ray from the eye to the gaze point. It is embedded within the cropped eye image. The six cropped eye images, derived from the combination of two eyes and three cameras, undergo processing by convolutional neural networks before being fused into the final multi-layer perceptron.

To enable the network to handle different image qualities, we design a weighted fusion mechanism. This mechanism utilizes a joint discriminator to assess the potential contribution of each cropped eye image and assigns lower weight to images with lower quality. This allows the network to prioritize valuable information for the final inference.

3.3.1 Design Motivation. The cropped eye images may sometimes have low quality and do not provide very useful information. For example, when the user blinks or temporarily close one of the eyes, as shown in Figure 4, the eye image may be relatively less helpful for gaze tracking task. In such cases, the system is expected to refer more to the image of the other eye. Additionally, for users that wear glasses, the light reflected by the glass and the glasses frames may also occasionally block the eye image and degrade its quality, as shown in Figure 4. In such cases, the current camera happens to capture the reflection or the glasses frames occluding the eye, while other cameras may have better angle of view.

In conclusion, to handle fluctuation of eye image quality caused by both eye and camera factors, each cropped image should be individually judged before being fused into the final decision layer. Images with lower quality and less important information should receive lower weights when being considered by the system.

(a) Blink (b) Eye closed (c) Reflection (d) Occlusion

Figure 4: Cropped eye image quality fluctuations

3.3.2 Weighted Fusion Network Design. To that end, we design an eye image discriminator to judge the quality of the cropped eye images, and their potential importance to the gaze tracking task. Based on the judgement, we conduct weighted fusion for the six information channels. The lower part of Figure 2 show the structure of the weight fusion design.

Specifically, besides the main convolutional neural network, we also input the cropped eye images into the discriminator to assign a weight for each image (w_1, w_2, and w_3 in Figure 2). The weights will pass through a softmax layer to be jointly normalized, and then multiplied onto the main convolutional neural network output, so that images with lower quality or importance will be considered less in the final decision multi-layer perceptron. The discriminator consists of two convolutional neural network layers and two fully connected layers, and outputs a 1-D scalar weight.

For simplicity, Figure 2 only depicts the processing of the right eye, while the cropped left eye images will also share the same discriminator to output the corresponding weights: w_4, w_5, and w_6. Additionally, $w_1 \sim w_6$ will all be fused into one softmax layer to output the jointly normalized weights.

3.4 Low-Effort Re-calibration via Implicit Data Collection at Click Opportunities

3.4.1 Design Motivation. In the real application of gaze tracking, users are typically asked to calibrate the system before using it. This is mainly due to biological difference, such as different eye sizes, shapes, and users' gaze habit. The calibration process usually requests the user to look at a visual target rendered on the screen, meanwhile, the cameras capture the user's eye image, allowing the system to acquire an aligned data sample. The aligned sample stores a valuable input-output pair. The input stands for the eye image, and the output means the gaze point, i.e., the pixel coordinate of the target rendered on the screen. Since the aligned data sample is captured while the user is staring at the target, this input-output pair can effectively supervise the calibration of the gaze tracking system.

However, such data collection requires active user attention, and thus can be laborious and expensive. Furthermore, learning-based algorithms typically require heavy training data to optimize the neural networks. In the worst scenario, the user may need to stare at target after target for a long time before the system is ready to use, which could cause eye fatigue and dissatisfaction. To avoid that, we propose to relieve the requirement of explicit user attention, by identifying implicit alignment opportunities during daily computer use.

3.4.2 Implicit Data Collection Module Design. There are visual events that could attract or imply user's attention, e.g., video saliency [36, 37]. Similarly, we look for such events in mouse clicking [39, 40]. As a most common tool for human-computer interaction, electronic mouse controls the cursor that navigates within the screen. The coordinate of the cursor on the screen can also be conveniently acquired.

When computer users click on buttons on the screen (e.g., the red-cross button that closes the window), they often use their sight to guide the cursor to navigate onto the button, and confirm its location before performing the click. During the click, the cursor can be temporarily considered as a visual target. As clicking is frequently iterated in daily computer using, it provides a great amount of aligned opportunities, where the user's gaze will align with, or approach the cursor. Collecting data during these valuable opportunities does not explicitly require user's active attention, and thus could potentially reduce the calibration overhead.

However, it is also important to filter the implicitly collected data to guarantee data quality, as users may not look at the cursor during clicking in certain cases. To examine the reliability of collecting aligned data during clicking, we recorded a total of 6.89 hours real-life active computer usage history from five users (user #16 $\sim$ #21), and summarize three major unaligned cases, as well as corresponding criteria that can help judge and refine the data collection quality.

Note that, we are now trying to evaluate whether the gaze point is aligned with the cursor's location, while the users are not explicitly instructed to look at anything, making it impossible to acquire the gaze point ground truth. Therefore, we use the inference of Tobii Pro Spark eye tracker as the provisional ground truth of the gaze point to to assess the extent of alignment between the cursor and the gaze point during clicking. Furthermore, we only consider left clicks, as they are the primary mode of interaction using a mouse.

- Criterion A: active application context:
  The primary factor that affects whether a user will look at the cursor during click is the application being used. In casual applications like file managing (file explorer, folder), internet browsing (Internet Explorer, Google Chrome, Fire Fox, not including online video web pages), and text editing (Notepad, Microsoft Doc., coding), gaze usually aligns well with the cursor when clicking. However, in applications such as games, the gaze point is almost never aligned with the cursor at all, as users are focusing on the visual feedback from the game. Additionally, many games hide the cursor during playing, particularly in first-person shooter games. Watching videos and movies also does not guarantee an alignment between the gaze and cursor, as users may randomly click anywhere on the video only to pause or resume playing, without looking at the cursor.
  Figure 5 illustrates the average alignment error (the distance between the gaze point and the cursor location during clicking) across different active application contexts. According to the real-life computer usage record, gaming and watching videos are found to be entirely unreliable, whereas file managing (folder), internet browsing, and text editing still offer opportunities for implicit data collection. Therefore, we only select these three contexts as the scope of implicit data collection. Within these three contexts, we are able to

filter out 15526 click opportunities, with an average alignment error of 4.90 cm. Building on this basic application context filtering, we then design criteria B and C to further improve the alignment accuracy.

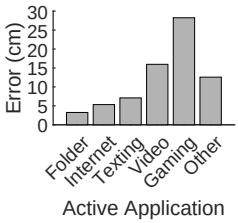

Figure 5: Error in different contexts

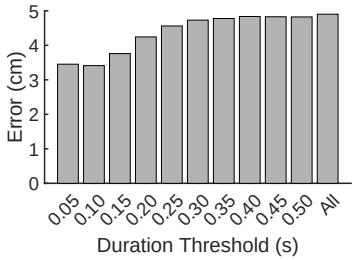

Figure 6: Error with different click duration threshold

- Criterion B: press-release duration:
  Clicking involves both pressing and releasing actions on the mouse. A click is identified when a release action immediately follows a press action. However, a press-release action pair can also occur during dragging, where the duration between the press and release actions is longer. Users typically engage in dragging actions during file management (throwing a file into another window), and text editing (selecting a sentence). In these cases, users will only vaguely confirm the cursor's destination, and the duration between the press and release is long (e.g., $0.2s$). Longer duration implies higher likelihood of the user's attention being diverted to other contents before releasing the mouse (e.g., dragging scroll bar, while checking web contents).
  We filter the click data samples using different press-release duration thresholds, as illustrated in Figure 6. According to the real usage record, imposing a limited duration threshold helps reduce the average alignment error. With a duration threshold of 0.1 second, we filtered the 15,526 samples down to 2,434, resulting in an average alignment error of 3.41 cm. However, further reducing the threshold to 0.05s does not necessarily reduce the error anymore, since users may occasionally click randomly with the mouse. As a result, we adopt the 0.1 second threshold as criterion B.

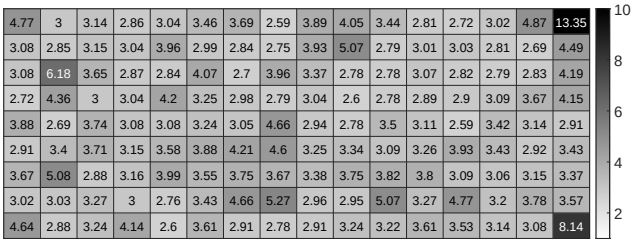

Figure 7: Spatial distribution of alignment error

- Criterion C: click location:
  Last, we examine the spatial distribution of the alignment error. Figure 7 shows the error distribution on the entire screen. The upper-right and bottom-right corners exhibit significantly higher errors, suggesting that users often do not precisely confirm the cursor location when clicking.
  As an explanation, skilled users may often heavily throw the cursor to the upper-right corner and click, as they know the close button is at that corner when the active window is maximized. The cursor being throttled to the upper-right corner will be confined by both the right and upper borders, and definitely land on the close button. Consequently, users only need to observe the window close as confirmation after they click the mouse, and do not need to look at the cursor. Similar rationale applies for the bottom-right corner, where the Windows system has a 'show-desktop' button that minimizes all windows and displays the desktop. However, this feature is less frequently used than the closed button.
  We use a Windows 11 system as the data collection platform, whose menu button is at the middle of the bottom taskbar. However, for Windows 10 and earlier versions, the menu button is positioned at the bottom-left corner, which may encounter similar issues if the system is to be deployed on these platforms. Additionally, Mac OS also features a menu button on the upper-left.
  We discard clicks that land within the range of 100 pixels from the four corner. As a result, 2,381 samples are filtered from the original 2,434 samples. The final average alignment error for the remaining 2,381 samples is 3.28cm.

With these criteria in place, we can effectively filter and refine the collected aligned clicks. During a recognized click opportunity, we capture the user's eye images via the camera networ. Along with recording the cursor location, we prepare an input-output pair as training data.

We save the captured eye images from the press moment till the release moment. Specifically, the cameras are activated when the mouse is pressed and continue capturing images for 0.1 second or until the mouse is released. According to criterion B, if the press-release duration exceeds 0.1 second, the click opportunity is recognized as invalid and discarded. Since human has a short reaction delay [43–45] after confirming the cursor location, we also record the eye images in the following 0.2 second after the release. This yields extra 6 data samples (webcam video recording rate is 30Hz).

For the remaining 2381 click opportunities, the average press-release duration is 0.0786 second. For each click opportunity, an average of 2.57 aligned data samples can be collected (e.g., a 0.01s duration can provide 1 sample at the press moment, and a 0.065s duration can only provide 2 samples, but 0.067s is able to provide 3 samples). Additionally, with the 0.2s recording after the click, and using the 6.89 hours of active computer usage, we can implicitly collect

20,405 data samples for training. The overall implicit data collection efficiency is 49.36 samples per minute.

To apply this module, the user simply operates the computer as usual, after setting up the cameras on the monitor. The module will collect the click opportunities and train the neural network at the backend. After a certain amount of time, the system will be ready to use. If the user is not satisfied with the implicitly calibrated result, they can always revert to classical calibration, where the user is explicitly required to actively stare at visual targets to collect gaze data.

Combining implicit and explicit data collection methods is also a viable option. For example, a user initially train the network implicitly with 30 minutes of daily computer usage data but finds the result unsatisfactory. In such cases, the user could spend an additional 3 minutes on explicit data collection, which originally would take 10 minutes of active staring that could cause eye fatigue, so that the implicit data collection module helped reducing the calibration effort.

4 EXPERIMENT

We collect gaze data and conduct real experiment to examine the gaze tracking performance of Tri-Cam. We mainly compare Tri-Cam to the state-of-the-art commercial eye tracker, Tobii Pro Spark, on various aspects.

4.1 Experiment Setting

4.1.1 Baselines.

- Tri-Cam: Our designed gaze tracking system, using three non-depth webcams. Tri-Cam uses cropped eye images only, which can be acquired from eye detection pre-processing. For post-processing, we conduct basic smoothing on the output of Tri-Cam to better fit the continuity of human gaze.
- Tobii Pro Spark eye tracker: The state-of-the-art commercial eye gaze tracking product developed by Tobii Technology, who focuses on providing eye-tracking solutions for academic and commercial research. Tobii Pro Spark features a a single integrated central infrared camera and two infrared transmitters on the sides. It is commonly used as gaze ground truth provider in other gaze tracking researches [23, 29], due to its high accuracy and robustness. However, we are using Tobii Pro Spark as a baseline for comparison.
- DV-Gaze [29]: The latest deep learning-based gaze tracking work that uses Resnet[46] and Transformer[47]. DV-Gaze uses double camera to capture face image, instead of eye images, to infer gaze. For DV-Gaze, we use the two webcams at the two sides as its input.
- AsGaze [23]: A geometry-based gaze tracking solution using a single non-depth RGB camera. AsGaze detects the iris boundary to estimate gaze ray. It can be configured to track gaze on different surface areas to suit various applications. We use the central camera as its input. However, its single-camera nature does not support free movement. Its error increases linearly as user moves away from the initial position. Therefore we only demonstrate its performance in the gaze angle section (Section 4.2.2).

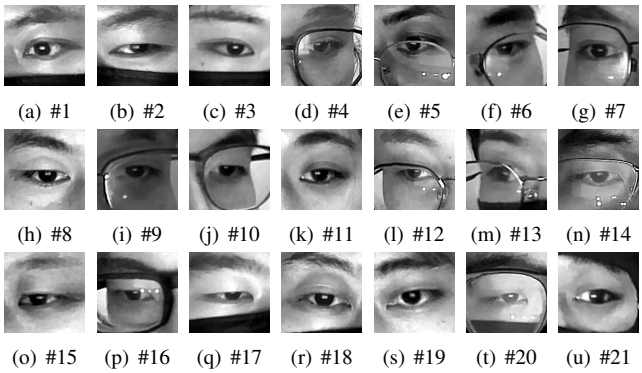

Figure 8: All users' eye image

4.1.2 Data Collection.

Existing datasets do not specifically study the impact of free movement, i.e., changing pose, moving body, and turning head. Additionally, they use Tobii as ground truth, which we are looking to compare with. To build practical gaze tracking system that allows free movement, we collect new dataset from 21 users. Among them, 11 wear glasses, 7 are female, and 14 are male. For every user we collect 4000 aligned data samples, using visual targets as gaze point ground truth. All users are allowed to blink, move body, and turn head. They are only asked to maintain a healthy distance of 45~60cm away from the monitor.

Three cameras are evenly deployed on the top of the monitor, facing front to capture eye images, while the Tobii Pro Spark eye tracker will simultaneously infer the gaze point.

Before each data collection session, we always ask the user to calibrate the Tobii Pro Spark using its standard calibration procedure provided by Tobii Tracker Manager software.

The time limit for each data collection session is 30 seconds, after which the user will be asked to take a break for a least one minute to ensure precise gazing. Sessions will be repeated till 4000 samples are collected.

4.1.3 Evaluation Metric.

The gaze tracking error is defined as the Cartesian distance between the estimated gaze point and the visual target, both of which refer to a specific pixel locations on the monitor. We convert the pixel distance to centimeters using the dimensions and resolution information of the monitor.

4.1.4 Hardware.

As shown in Figure 9, we use the 27" Dell SE2719H as the main monitor for experiment. Its resolution is 1920×1080, and dimensions are $59.789 \times 33.631 cm$. The Tobii Pro Spark Eye Tracker is installed at the bottom of the screen, facing front. We use three Svarog webcams as camera

Figure 9: Monitor, webcams, and Tobii Pro Spark

input, whose video resolution is 1920×1080, and recording rate is 30Hz. They are evenly placed at the top of the Dell monitor, facing forward. For neural network model training, we use an Alienware M18 laptop, which is equipped with a NVIDIA® GeForce RTX™ 4090, 16 GB GDDR6 graphic card, and 24-Core Intel® Core™i9 13980HX Processor. We also put the webcams on the Alienware M18 laptop monitor, and use it as a second screen for experiment. Its resolution 1920×1200, and dimensions are $38.776 \times 24.235cm$.

4.2 Overall Performance

The overall gaze estimation error for Tri-Cam, Tobii, and DV-Gaze are 2.0558cm, 1.9455cm, and 3.7077cm, respectively. Although Tri-Cam has higher error than Tobii, it reaches the similar level of accuracy using three inexpensive webcams.

4.2.1 Different Users. As personal computers are usually owned by individuals, a gaze tracking system should also be trained using personal gaze data, since it will also be mainly used by the individual. Therefore, for user #1 $\sim$ #16, we train a model for each user individually, and test the gaze tracking accuracy on the user's own test data. Specifically, we allocate 70% of the data (2,800 samples) for training, 10% (400 samples) for validation to select the best model during training, and 20% (800 samples) as unseen data for testing.

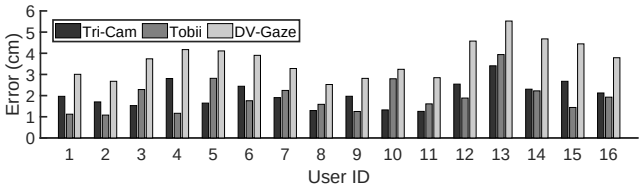

Figure 10: Gaze tracking error on different users

As shown in Figure 10, Tri-Cam has an overall similar performance to Tobii. The average gaze estimation error for Tri-Cam, Tobii, and DV-Gaze are 2.0558cm, 1.9455cm, and 3.7077cm, respectively. Although Tri-Cam and DV-Gaze share the same webcams as input, DV-Gaze is designed to use two cameras, while Tri-Cam uses three, which potentially provides more visual details. Additionally, DV-Gaze uses the face image, instead of cropped eye image, which may introduce more visual influence to the convolutional neural networks, e.g., facial landmarks like nose and mouth will also be considered by the neural network, while having little relevance to the gaze inference. Tri-Cam, on the other hand, focuses only on the cropped eye images, which have concentrated information relevant to gaze tracking, and thus could potentially learn and perform better.

Compared to Tobii, Tri-Cam has an overall similar performance. For certain users, the error of Tobii eye tracker is a lot higher, which is possibly due to the eyes' biological differences. Tobii eye tracker uses infrared signal to track the gaze, as human eyes have special interaction with the infrared, which is dependent on the eyes' biological nature. Therefore, even though it has been calibrated by the user before using, Tobii could still have fluctuating accuracy across different users. Additionally, for users that wear glasses, their glasses are made of different materials, among which, resin glasses are usually transparent to infrared, while glass-based materials could block the infrared spectrum, making Tobii inaccurate. On the other hand, Tri-Cam is relatively consistent across users, as we only use the visible light spectrum.

4.2.2 Different Gaze Angles. We also test the accuracy of Tri-Cam at different view angles. This angle refers to the angle between the user's gaze direction and the perpendicular direction of the monitor screen. While maintaining a distance of 50cm from the monitor, we set a horizontal offset for the users' seat location, starting at the central position facing the monitor. Different gaze angle θ results in different offset Δx:

$$\theta \in \{-30°, -20°, -10°, -5°, 0°, 5°, 10°, 20°, 30°\} \quad (1)$$

$$\Delta x = L \cdot tan(\theta) \qquad (L = 50cm) \quad (2)$$

Figure 11 shows the average gaze estimation error at different gaze angle, as well as the detection rate of Tobii, denoted on each bar that represents Tobii's performance. 0° means the user sits along the central axis of the monitor, where Tobii is very accurate and robust. Its error at 0° is 1.0049cm , and detection rate is 95.5%. However, as soon as the user move to ±5°, Tobii's detection rate begins to drop, and error begins to rise. If the absolute gaze angle exceeds 10°, Tobii is unable to detect the eyes and provide gaze inference. This is highly possibly due to the single-camera configuration of Tobii. Although the infrared camera of Tobii is powerful at gaze tracking, it could still only cover limited area from a single origin. AsGaze has an error of 1.8091cm at 0°, but intensively increases along view angle. Single-non-depth-camera systems

cannot handle free movement. It is only accurate at its initial position, namely, 0°, where it was calibrated.

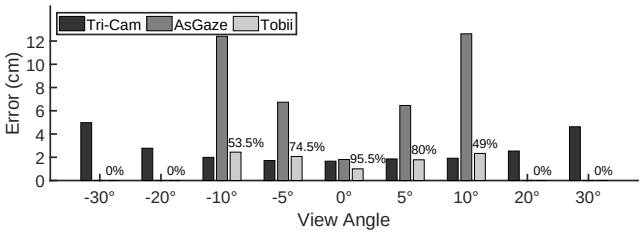

Figure 11: Performance at different view angles

On the other hand, since Tri-Cam utilizes three cameras, it is always able to detect the eyes in the view of at least two of its cameras. The error of Tri-Cam also rises as the absolute gaze angle increases, but in a slower trend. We notice that at ±30° view angle, there would be one camera that cannot capture the user in its view, thus Tri-Cam will have to rely on the rest of the two cameras. Therefore, the error of Tri-Cam largely increases to around 5cm. However, looking at the screen with a ±30° angle is not comfortable or common, and Tri-Cam is able to maintain a relatively stable performance within the range of $[-20°, 20°]$. This indicates the potential of Tri-Cam to support free-movement gaze tracking.

4.2.3 Spatial Distribution of Accuracy. Figure 12 shows the spatial distribution of the gaze estimation error of Tri-Cam. The overall error distribution is relatively even. The central area has the overall best accuracy, since the user is directly facing the monitor when looking at the center of it. The camera network receives maximum image quality at this angle. The performance on borders and corners are also acceptable, as Tri-Cam utilizes three cameras, each one of which serves as backup for the others, and are usually able to find a valid angle of view to the eyes.

2.54	3	2.05	1.84	2.05	1.72	1.54	1.33	1.78	2.17	2.36	2.68	1.5	2.29	2.48	1.69
2.07	2.32	2.85	1.69	2.09	1.71	1.7	2.53	2.47	2	2.12	1.67	2.5	2.54	1.96	1.94
1.32	3.12	3.64	1.37	1.32	1.53	1.5	3.05	2.09	1.32	2.06	1.61	1.84	2.11	1.69	1.35
1.35	2.56	3	1.17	1.97	3.31	2.1	1.55	1.88	1.55	1.98	1.94	2.65	4.22	1.97	1.58
1.11	1.61	3.86	2.1	1.57	1.77	1.68	1.25	1.73	1.13	1.84	2.63	2.61	4.18	1.94	2.11
1.18	1.69	2.5	3.25	1.98	2.3	1.53	2.21	2.41	1.99	1.69	2.24	2.27	2.27	1.89	1.56
1.06	3.19	2.88	1.9	1.65	1.75	2.18	1.66	1.89	1.53	1.56	2.76	3.2	3.35	1.88	1.42
1.47	2.42	1.77	2.1	3.14	2.66	2.8	1.57	1.66	1.59	1.98	1.84	2.02	2.38	2.05	1.61
1.99	2.13	2.32	2.31	2.55	2.18	2.21	2.33	1.39	1.47	2.48	1.52	1.61	2.4	2.4	2.44

10
8
6
4
2

Figure 12: Spatial distribution of gaze tracking error

However, there are two specific clusters that have relatively high error, located around $(300, 480)$ and $(1600, 480)$. As a possible explanation, for users who wear glasses, looking at these two coordinates will reflect more light from the screen to the camera on either left or right side. These two areas are slightly above the middle line, since the cameras are deployed on the top of the monitor, and would receive the reflection when the user is looking slightly above. Assume the user is looking at the $(1600, 480)$ area. The reflected light will block the eye image and make the right camera unreliable. Therefore, Tri-Cam has to focus more on the middle and left cameras, which would have an angled view to the user's eye, especially for the left camera, as the user is looking to the right side. On the other hand, if the user is looking at the center and the reflected light is blocking the vision of the middle camera, both left and right cameras can serve as backup, and have better angle of view, compared to the side situations. Therefore the middle area does not have a serious reflection issue, which is also possibly due to the compensation of the high image quality at this angle.

4.2.4 Unseen Users. To test the scalability of Tri-Cam, we train a general model using all training data from user #1 ∼ 16. The average error of the general model on the test data of user #1 ∼ 16 is 3.0031cm. Then, we test the general model on five unseen users (#17 ∼ 21). As shown in Figure 13, the general model performs very bad on the five unseen users, especially for user #20, who wears glasses. The average error among the five users is 10.2674 cm, which is not capable of supporting gaze tracking applications. If the model is trained with the user's personal training data, it can perform relatively well, and the average error among the five users is 1.8918 cm. On the unseen users' data, Tobii still has a stable performance, with an average error of 1.6864 cm.

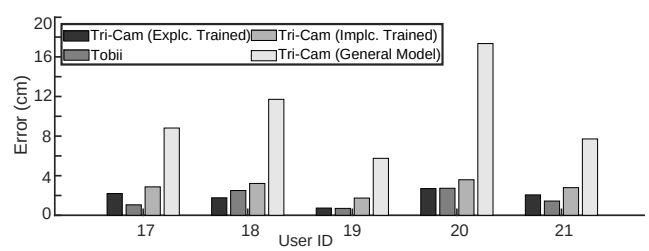

Figure 13: Performance on unseen users

This result indicates that Tri-Cam does not perform well without calibration (i.e., being trained with new users' data). The main reason lies in the difference of appearance among users. As shown in Figure 8, the size and shape or the eyes, as well as facial feature around the eyes, vary largely among users. Additionally, the gaze pattern of users may also be biologically different. Therefore, it could be difficult for the convolutional neural network to know how to track the gaze of unseen users. Note that, the Tobii Pro Spark eye tracker is always calibrated with the new user's participation before being used, while this condition is missing for Tri-Cam, as it is not trained with new users' data. Scalability has been a popular topic for neural network design. However, we leave the possibility of improving it with further complex design to

future works. Instead, we rely on the implicit re-calibration module to adapt to new users' feature.

Implicit re-calibration on unseen user. New users can always re-calibrate Tri-Cam via performing explicit data collection and re-train the Tri-Cam model. It is also viable to use the implicit data collection module and re-calibrate Tri-Cam without having to stare at visual targets. Therefore we also show the performance of Tri-Cam trained with implicitly collected data from the five unseen users. From Figure 13, we can see that the error of the implicitly trained model is higher than the explicitly trained model. This is due to the alignment error of the implicitly collected click samples. According to the supervision of Tobii, the expectation of the alignment error of the click samples is 3.28cm, after all three criteria have been applied. With these sub-accurate training data, Tri-Cam reaches an average error of 2.8087 cm, which is 48.47% higher than the explicitly trained model. This increase is much smaller than the alignment error of 3.28cm, which is mainly due to two reasons. First, the alignment error is evaluated using Tobii as temporary ground truth, which may not be absolutely accurate. Second, the difference vector of from the implicitly collected click samples to the actual gaze point during the click could be in various directions, and thus, aggregating them could potentially decrease the overall alignment error. Therefore, in the batched training of Tri-Cam using the implicitly collected data, the impact of the alignment error could be lower than the absolute value of the alignment error. In conclusion, the implicit data collection module is capable of training Tri-Cam to a applicable level of accuracy, without having to ask the user to explicitly stare at visual targets to collect gaze data. This module is also capable of supporting low-effort re-calibration of the model in the case of new monitor-camera setup.

4.3 Training Efficiency and Overhead

We now demonstrate the training efficiency of Tri-Cam. For the 4000 data samples collected by each user, 2800 of them are used for training, and 700 are used for verification. Timer shows that training with all 2800 training data takes 2.1305 seconds per one epoch. On the other hand, DV-Gaze takes 32.2898 seconds per epoch. This difference is due to the neural network size and complexity. Compared to the tiny cropped images used by Tri-Cam, DV-Gaze uses the face image as input, which demands larger convolutional neural networks to handle the larger image input. DV-Gaze also uses both Resnet and Transformer, which are known to be computationally intensive. From the statistical aspect, Tri-Cam has 1245113 parameters, while DV-Gaze has 13619382 parameters. Higher complexity results in larger optimization overhead. Tri-Cam divides the camera-eye geometry and the eye-screen geometry, and uses relatively lightweight neural networks to handle them, resulting in faster training.

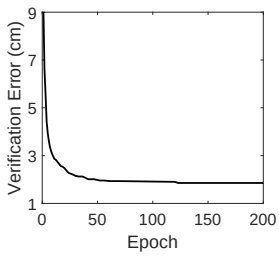

Figure 14: Verification error during training

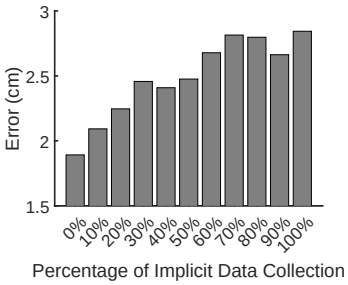

Figure 15: Combining explicit and implicit Data collection

Figure 14 also shows the model verification error of Tri-Cam during training, averaged among users. We can see that the model's verification error converges after 150 training epochs, which is within 6 minutes. This guarantees that the user do not need to wait long for Tri-Cam to be ready to use.

Combining Explicit and Implicit Data Collection. Beside total explicit data collection and total implicit data collection, it is also viable to combine the implicit and explicit data collection, if the user is not satisfied with the with the implicitly trained model. This could potentially reduce the overhead of data collection and avoid eye fatigue. To evaluate the effect of the combination of explicit and implicit data collection, among the 2800 explicitly collected training data samples, we replace certain percentage of them with equal amount of data samples implicitly collected from the same user. With different percentage setting, we train a individual model, and test its gaze tracking accuracy. Figure 15 shows the average error of each model tested on user #17 $\sim$ #21. We can see as the percentage of implicit data collection increases, the error rises accordingly. It is because the implicitly collected data has higher alignment error by its nature, i.e., the user's gaze does not completely align with the visual target, while the visual target's location is being used as gaze point ground truth. We also notice that when at a low percentage (<30%), the percentage itself has a greater impact on the model accuracy. As a speculation, small portion of implicitly collected data is enough to effectively pollute the data pool, confusing the model during training. This implies that it is preferred to either use implicit collection more (>30%), or not use it at all, when in combination with explicit data collection.

4.4 Ablation Study

We examine the contribution of each system component, via cutting it off and test the performance of Tri-Cam.

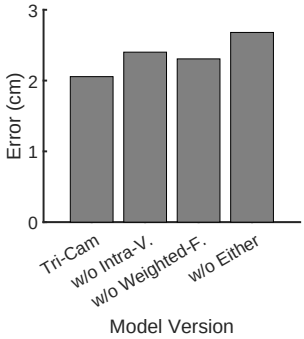

Figure 16: Design component ablation study

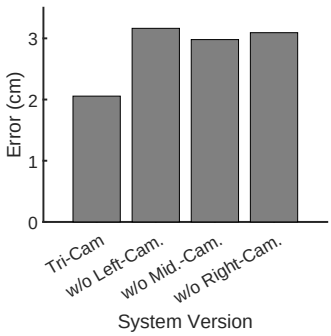

Figure 17: Importance of each camera

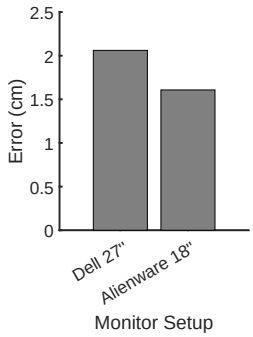

Figure 18: Different monitor setups

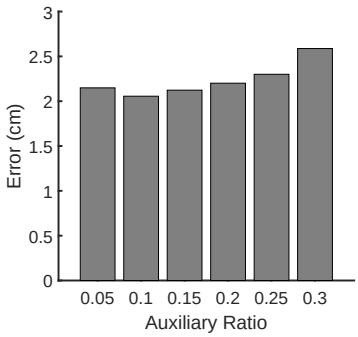

Figure 19: Influence of auxiliary ratio

4.4.1 Cutting off Intra-validation. We first cut off the intra-validation component, and the gaze tracking error of Tri-Cam rises to 2.4015cm, which is 16.82% higher than the complete model. It shows that the auxiliary multitasking is capable of supporting the training of the main model, enabling extra back propagation via the auxiliary outputs. The intra-validation design does not introduce any other information or measurement into the system, but instead, exploits the intra-relationship between cameras' view angle to improve the performance of Tri-Cam.

4.4.2 Cutting off Weighted Fusion. Second, cutting off the weighted-fusion design increases the error of Tri-Cam to 2.3065cm, which is 12.19% higher. This indicates that the discriminator is capable of judging the cropped eye images before they are considered in the final decision layer, and granting weights to the images, so that the neural network can focus less on images with lower quality.

4.4.3 Cutting off One Camera. Last, to examine the importance of each camera, we cut off one camera at a time. As shown in Figure 17, all three cameras are important to Tri-Cam. Without any of three cameras, the error rises to around 3cm, due to the missing support from the intra-validation mechanism. Cutting off one of the side cameras appears to be slightly more deadly to Tri-Cam than cutting off the middle camera. The distance between the side cameras is twice the distance between the middle camera and either of the side cameras, which could potentially support better depth perception, and thus makes the version without the middle camera slightly better than the others.

4.5 Other Factors of Interest

4.5.1 Different Monitor-Camera Setup. We also setup the webcams on the Alienware laptop and use its 18" screen as a second monitor for experiment. User #21 collected data on this laptop, and thus we only compare the results of user #21 on the two monitor setups. As shown in Figure 18, the gaze tracking error is 2.0610cm and 1.6075cm for the 27" and 18" monitor setups, respectively. First, it demonstrates that Tri-Cam can be applied on different monitor setups. Second, the decrease in the gaze tracking error is probably due to the confined space of possible gaze point. Specifically, the area of the 1920×1080-27" screen is 2010.76 cm^2, while the 1920×1200-18" screen is 939.74 cm^2, which is approximately half of the 27" screen's area. It is similar to cutting a half from the 27" screen, and only use the gaze data within this area for training and testing, which makes the task simpler.

4.5.2 Inference Latency. Statistics show when running on the Alienware laptop, Tri-Cam takes 0.1741ms to infer one gaze point, which ensures that the maximum real time inference rate of Tri-Cam is 5745Hz. Although other platforms may have less computation power, this result is still more than enough to support the 30Hz webcams as input. This is mainly due to the lightweight network design of Tri-Cam. In potential real application, it is very unlike that the network inference time of Tri-Cam would be the bottleneck, when collaborating with other components, such as webcam recording, eye detection algorithm, gaze result rendering, etc..

4.5.3 Auxiliary Ratio. We set 0.1 as the auxiliary multitasking ratio, which will be multiplied onto the auxiliary loss before it is incorporated into the joint loss. We also test the performance of Tri-Cam with different choices of the auxiliary ratio. Figure 19 shows the gaze tracking error of Tri-Cam with different auxiliary ratio settings. We can see that 0.1 provides the best accuracy. Lower auxiliary ratio will eliminate the effect of intra-validation via auxiliary multitasking, while higher auxiliary ratio will over stress the intra-validation, particularly given that there are six auxiliary outputs.

5 CONCLUSION

This paper introduces Tri-Cam, a practical deep learning-based gaze tracking system using three affordable RGB webcams, featuring a split network structure for efficient training,

as well as designated network designs to handle the separated gaze tracking tasks. An implicit calibration module is deployed to make use of mouse click opportunities to reduce calibration effort for the user. Experiment shows Tri-Cam achieves comparable accuracy to Tobii, the state-of-the-art commercial eye tracker, while supporting a wider free movement area. In conclusion, Tri-Cam provides a user-friendly, affordable, and robust gaze tracking solution that could practically enable various applications.

REFERENCES

[1] Yili Ren, Zi Wang, Sheng Tan, Yingying Chen, and Jie Yang. Winect: 3d human pose tracking for free-form activity using commodity wifi. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 5(4):1–29, 2021.

[2] Zhe Chen, Chao Cai, Tianyue Zheng, Jun Luo, Jie Xiong, and Xin Wang. Rf-based human activity recognition using signal adapted convolutional neural network. *IEEE Transactions on Mobile Computing*, 22(1):487–499, 2021.

[3] Fusang Zhang, Kai Niu, Jie Xiong, Beihong Jin, Tao Gu, Yuhang Jiang, and Daqing Zhang. Towards a diffraction-based sensing approach on human activity recognition. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 3(1):1–25, 2019.

[4] Yongsen Ma, Sheheryar Arshad, Swetha Muniraju, Eric Torkildson, Enrico Rantala, Klaus Doppler, and Gang Zhou. Location-and person-independent activity recognition with wifi, deep neural networks, and reinforcement learning. *ACM Transactions on Internet of Things*, 2(1):1–25, 2021.

[5] Preeti Vyas, Unma Mayur Desai, Karin Yamakawa, and Karon Maclean. A descriptive analysis of a formative decade of research in affective haptic system design. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*, CHI '23, New York, NY, USA, 2023. Association for Computing Machinery.

[6] Zeqi Lai, Y. Charlie Hu, Yong Cui, Linhui Sun, and Ningwei Dai. Furion: Engineering high-quality immersive virtual reality on today's mobile devices. In *Proceedings of the 23rd Annual International Conference on Mobile Computing and Networking*, MobiCom '17, page 409–421, New York, NY, USA, 2017. Association for Computing Machinery.

[7] Jonathan Liebers and Stefan Schneegass. Gaze-based authentication in virtual reality. In *ACM Symposium on Eye Tracking Research and Applications*, ETRA '20 Adjunct, New York, NY, USA, 2020. Association for Computing Machinery.

[8] Brendan David-John, Candace Peacock, Ting Zhang, T. Scott Murdison, Hrvoje Benko, and Tanya R. Jonker. Towards gaze-based prediction of the intent to interact in virtual reality. In *ACM Symposium on Eye Tracking Research and Applications*, ETRA '21 Short Papers, New York, NY, USA, 2021. Association for Computing Machinery.

[9] Andrew T Duchowski and Andrew T Duchowski. *Eye tracking methodology: Theory and practice*. Springer, 2017.

[10] Päivi Majaranta and Andreas Bulling. Eye tracking and eye-based human–computer interaction. In *Advances in physiological computing*, pages 39–65. Springer, 2014.

[11] Dario D Salvucci and Joseph H Goldberg. Identifying fixations and saccades in eye-tracking protocols. In *Proceedings of the 2000 symposium on Eye tracking research & applications*, pages 71–78, 2000.

[12] Ali Borji and Laurent Itti. Defending yarbus: Eye movements reveal observers' task. *Journal of Vision*, 14(3):29–29, 2014.

[13] Kenneth Holmqvist, Marcus Nyström, Richard Andersson, Richard Dewhurst, Halszka Jarodzka, and Joost van de Weijer. *Eye Tracking: A Comprehensive Guide to Methods and Measures*. Oxford University Press, 2011.

[14] Mohamed Khamis, Mariam Hassib, Emanuel von Zezschwitz, Andreas Bulling, and Florian Alt. Gazetouchpin: protecting sensitive data on mobile devices using secure multimodal authentication. *Proceedings of the 19th acm international conference on multimodal interaction*, pages 446–450, 2017.

[15] Mohamed Khamis, Axel Hoesl, Alexander Klimczak, Martin Reiss, Florian Alt, and Andreas Bulling. Eyescout: Active eye tracking for position and movement independent gaze interaction with large public displays. In *Proceedings of the 30th Annual ACM Symposium on User Interface Software and Technology*, UIST '17, page 155–166, New York, NY, USA, 2017. Association for Computing Machinery.

[16] Omair Shahzad Bhatti, Michael Barz, and Daniel Sonntag. Eyelogin - calibration-free authentication method for public displays using eye gaze. In *ACM Symposium on Eye Tracking Research and Applications*, ETRA '21 Short Papers, New York, NY, USA, 2021. Association for Computing Machinery.

[17] Simon P Liversedge and John M Findlay. Saccadic eye movements and cognition. *Trends in cognitive sciences*, 4(1):6–14, 2000.

[18] Roy S Hessels and Ignace T Hooge. Eye tracking in developmental cognitive neuroscience—the good, the bad and the ugly. *Developmental Cognitive Neuroscience*, 40:100710, 2019.

[19] Kenneth Holmqvist and R Andersson. Eye tracking: A comprehensive guide to methods. *Paradigms and measures*, 2017.

[20] Eunji Chong, Katha Chanda, Zhefan Ye, Audrey Southerland, Nataniel Ruiz, Rebecca M Jones, Agata Rozga, and James M Rehg. Detecting gaze toward eyes in natural social interactions and its use in child gaze analysis. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 1(3):1–20, 2017.

[21] Ulrich J Pfeiffer, Kai Vogeley, and Leonhard Schilbach. From gaze cueing to dual eye-tracking: Novel approaches to investigate the neural correlates of gaze in social interaction. *Neuroscience & Biobehavioral Reviews*, 37(10):2516–2528, 2013.

[22] Aldert Vrij, Karen Edward, Karl P Roberts, and Ray Bull. Detecting deceit via analysis of verbal and nonverbal behavior. *Journal of Nonverbal Behavior*, 24(4):239–263, 2000.

[23] Jiani Cao, Chengdong Lin, Yang Liu, and Zhenjiang Li. Gaze tracking on any surface with your phone. In *Proceedings of the 20th ACM Conference on Embedded Networked Sensor Systems*, SenSys '22, page 320–333, New York, NY, USA, 2023. Association for Computing Machinery.

[24] Sinh Huynh, Rajesh Krishna Balan, and JeongGil Ko. imon: Appearance-based gaze tracking system on mobile devices. *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 5(4):1–26, 2021.

[25] MS Shashi Kumar, KS Vimala, and N Avinash. Face distance estimation from a monocular camera. In *2013 IEEE International Conference on Image Processing*, pages 3532–3536. IEEE, 2013.

[26] Li Sun, Zicheng Liu, and Ming-Ting Sun. Real time gaze estimation with a consumer depth camera. *Information Sciences*, 320:346–360, 2015.

[27] Xuehan Xiong, Zicheng Liu, Qin Cai, and Zhengyou Zhang. Eye gaze tracking using an rgbd camera: a comparison with a rgb solution. In *Proceedings of the 2014 ACM International Joint Conference on Pervasive and Ubiquitous Computing: Adjunct Publication*, UbiComp '14 Adjunct, page 1113–1121, New York, NY, USA, 2014. Association for Computing Machinery.

[28] Tobii Pro AB. Tobii eye tracker: The next generation of head tracking and eye tracking, engineered for pc gamers., 2016.

[29] Yihua Cheng and Feng Lu. Dvgaze: Dual-view gaze estimation. In *Proceedings of the IEEE/CVF International Conference on Computer*

Vision (ICCV), pages 20632–20641, October 2023.

[30] Marc Tonsen, Julian Steil, Yusuke Sugano, and Andreas Bulling. Invisibleeye: Mobile eye tracking using multiple low-resolution cameras and learning-based gaze estimation. *Proc. ACM Interact. Mob. Wearable Ubiquitous Technol.*, 1(3), sep 2017.

[31] Yoichiro Hisadome, Tianyi Wu, Jiawei Qin, and Yusuke Sugano. Rotation-constrained cross-view feature fusion for multi-view appearance-based gaze estimation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 5985–5994, January 2024.

[32] John Gideon, Shan Su, and Simon Stent. Unsupervised multi-view gaze representation learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5001–5009, 2022.

[33] Yoichiro Hisadome, Tianyi Wu, Jiawei Qin, and Yusuke Sugano. Rotation-constrained cross-view feature fusion for multi-view appearance-based gaze estimation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 5985–5994, 2024.

[34] Lukas Liebel and Marco Körner. Auxiliary tasks in multi-task learning. *arXiv preprint arXiv:1805.06334*, 2018.

[35] Aakriti Adhikari and Sanjib Sur. Argosleep: Monitoring sleep posture from commodity millimeter-wave devices. In *IEEE INFOCOM 2023-IEEE Conference on Computer Communications*, pages 1–10. IEEE, 2023.

[36] Songzhou Yang, Yuan He, and Meng Jin. vgaze: Implicit saliency-aware calibration for continuous gaze tracking on mobile devices. In *IEEE INFOCOM 2021-IEEE Conference on Computer Communications*, pages 1–10. IEEE, 2021.

[37] Songzhou Yang, Meng Jin, and Yuan He. Continuous gaze tracking with implicit saliency-aware calibration on mobile devices. *IEEE Transactions on Mobile Computing*, 2022.

[38] Xucong Zhang, Michael Xuelin Huang, Yusuke Sugano, and Andreas Bulling. Training person-specific gaze estimators from user interactions with multiple devices. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems*, CHI '18, page 1–12, New York, NY, USA, 2018. Association for Computing Machinery.

[39] Yusuke Sugano, Yasuyuki Matsushita, Yoichi Sato, and Hideki Koike. Appearance-based gaze estimation with online calibration from mouse operations. *IEEE Transactions on Human-Machine Systems*, 45(6):750–760, 2015.

[40] Michael Xuelin Huang, Tiffany CK Kwok, Grace Ngai, Stephen CF Chan, and Hong Va Leong. Building a personalized, auto-calibrating eye tracker from user interactions. In *Proceedings of the 2016 CHI conference on human factors in computing systems*, pages 5169–5179, 2016.

[41] Andrew Housholder, Jonathan Reaban, Aira Peregrino, Georgia Votta, and Tauheed Khan Mohd. Evaluating accuracy of the tobii eye tracker 5. In *International Conference on Intelligent Human Computer Interaction*, pages 379–390. Springer, 2021.

[42] Google mediapipe face landmarking. https://developers.google.com/mediapipe/solutions/vision/face_landmarker. Accessed: April 30, 2024.

[43] Michael S Gazzaniga, Richard B Ivry, and GR Mangun. *Cognitive Neuroscience. The biology of the mind,(2014)*. Norton: New York, 2006.

[44] Robert S Weinberg and Daniel Gould. *Foundations of sport and exercise psychology*. Human kinetics, 2023.

[45] Christopher D Wickens, William S Helton, Justin G Hollands, and Simon Banbury. *Engineering psychology and human performance*. Routledge, 2021.

[46] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016.

[47] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in neural information processing systems*, pages 5998–6008, 2017.